My Project

Generated by Doxygen 1.8.17

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 sprite_s Struct Reference . . . 5

3.1.1 Detailed Description . . . 5

3.1.2 Member Data Documentation . . . 5

3.1.2.1 h . . . 5

3.1.2.2 w . . . 5

3.1.2.3 x . . . 6

3.1.2.4 y . . . 6

3.2 textures_s Struct Reference . . . 6

3.2.1 Detailed Description . . . 6

3.2.2 Member Data Documentation . . . 6

3.2.2.1 background . . . 6

3.3 world_s Struct Reference . . . 7

3.3.1 Detailed Description . . . 7

3.3.2 Member Data Documentation . . . 7

3.3.2.1 gameover . . . 7

3.3.2.2 sprite . . . 7

4 File Documentation 9

4.1 main.c File Reference . . . 9

4.1.1 Detailed Description . . . 11

4.1.2 Function Documentation . . . 11

4.1.2.1 apply_background() . . . 11

4.1.2.2 apply_sprite() . . . 12

4.1.2.3 clean() . . . 12

4.1.2.4 clean_data() . . . 12

4.1.2.5 clean_textures() . . . 13

4.1.2.6 handle_events() . . . 13

4.1.2.7 init() . . . 13

4.1.2.8 init_data() . . . 14

4.1.2.9 init_sprite() . . . 14

4.1.2.10 init_textures() . . . 14

4.1.2.11 is_game_over() . . . 15

4.1.2.12 print_sprite() . . . 15

4.1.2.13 refresh_graphics() . . . 15

4.1.2.14 update_data() . . . 16

4.2 sdl2-light.c File Reference . . . 16

4.2.1 Detailed Description . . . 17
4.2.2 Function Documentation . . . 17
4.2.2.1 apply_texture() . . . 17
4.2.2.2 clean_sdl() . . . 18
4.2.2.3 clean_texture() . . . 18
4.2.2.4 clear_renderer() . . . 18
4.2.2.5 init_sdl() . . . 19
4.2.2.6 load_image() . . . 19
4.2.2.7 pause() . . . 19
4.2.2.8 update_screen() . . . 20
4.3 sdl2-light.h File Reference . . . 20
4.3.1 Detailed Description . . . 21
4.3.2 Function Documentation . . . 22
4.3.2.1 apply_texture() . . . 22
4.3.2.2 clean_sdl() . . . 22
4.3.2.3 clean_texture() . . . 22
4.3.2.4 clear_renderer() . . . 23
4.3.2.5 init_sdl() . . . 23
4.3.2.6 load_image() . . . 23
4.3.2.7 pause() . . . 24
4.3.2.8 update_screen() . . . 24

Index **25**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

sprite_s
: Type qui correspond au sprite 5

textures_s
: Représentation pour stocker les textures nécessaires à l'affichage graphique 6

world_s
: Représentation du monde du jeu 7

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

main.c
: Programme principal initial du niveau 0 9

sdl2-light.c
: Sur-couche de SDL2 pour simplifier son utilisation pour le projet 16

sdl2-light.h
: En-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet 20

Chapter 3

Class Documentation

3.1 sprite_s Struct Reference

Type qui correspond au sprite.

Public Attributes

- int x
- int y
- int w
- int h

3.1.1 Detailed Description

Type qui correspond au sprite.

3.1.2 Member Data Documentation

3.1.2.1 h

```
int sprite_s::h
```

Champ indiquant la hauteur h du sprite

3.1.2.2 w

```
int sprite_s::w
```

Champ indiquant la largeur w du sprite

3.1.2.3 x

```
int sprite_s::x
```

Champ indiquant la position x du sprite

3.1.2.4 y

```
int sprite_s::y
```

Champ indiquant la position y du sprite

The documentation for this struct was generated from the following file:

- main.c

3.2 textures_s Struct Reference

Représentation pour stocker les textures nécessaires à l'affichage graphique.

Public Attributes

- SDL_Texture * background
- SDL_Texture * **sprite**

3.2.1 Detailed Description

Représentation pour stocker les textures nécessaires à l'affichage graphique.

3.2.2 Member Data Documentation

3.2.2.1 background

```
SDL_Texture* textures_s::background
```

Texture liée à l'image du fond de l'écran.

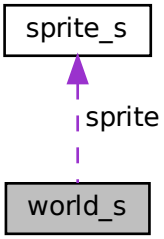
The documentation for this struct was generated from the following file:

- main.c

3.3 world_s Struct Reference

Représentation du monde du jeu.

Collaboration diagram for world_s:



Public Attributes

- sprite_t sprite
- int gameover

3.3.1 Detailed Description

Représentation du monde du jeu.

3.3.2 Member Data Documentation

3.3.2.1 gameover

```
int world_s::gameover
```

Champ indiquant si l'on est à la fin du jeu

3.3.2.2 sprite

```
sprite_t world_s::sprite
```

Champ représentant le sprite

The documentation for this struct was generated from the following file:

- main.c

Chapter 4

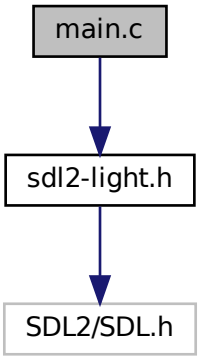
File Documentation

4.1 main.c File Reference

Programme principal initial du niveau 0.

```
#include "sdl2-light.h"
```

Include dependency graph for main.c:



Classes

- struct sprite_s

  Type qui correspond au sprite.
- struct textures_s

  Représentation pour stocker les textures nécessaires à l'affichage graphique.
- struct world_s

  Représentation du monde du jeu.

Macros

- #define SCREEN_WIDTH 300

  Largeur de l'écran de jeu.
- #define SCREEN_HEIGHT 480

  Hauteur de l'écran de jeu.
- #define SHIP_SIZE 32

  Taille d'un vaisseau.
- #define METEORITE_SIZE 32

  Taille d'un météorite.
- #define FINISH_LINE_HEIGHT 10

  Hauteur de la ligne d'arrivée.
- #define MOVING_STEP 10

  Pas de déplacement horizontal du vaisseau.
- #define INITIAL_SPEED 2

  Vitesse initiale de déplacement vertical des éléments du jeu.

Typedefs

- typedef struct sprite_s sprite_t

  Type qui correspond au sprite.
- typedef struct textures_s textures_t

  Représentation pour stocker les textures nécessaires à l'affichage graphique.
- typedef struct world_s world_t

  Représentation du monde du jeu.

Functions

- void print_sprite (sprite_t ∗sprite)

  Fonction de test des coordonnées du sprite.
- void init_data (world_t ∗world)

  La fonction initialise les données du monde du jeu.
- void clean_data (world_t ∗world)

  La fonction nettoie les données du monde.
- int is_game_over (world_t ∗world)

  La fonction indique si le jeu est fini en fonction des données du monde.
- void update_data (world_t ∗world)

  La fonction met à jour les données en tenant compte de la physique du monde.
- void handle_events (SDL_Event ∗event, world_t ∗world)

  La fonction gère les évènements ayant eu lieu et qui n'ont pas encore été traités.
- void clean_textures (textures_t ∗textures)

  La fonction nettoie les textures.
- void init_sprite (sprite_t ∗sprite, int x, int y, int w, int h)

  La fonction initialise la position du sprite.
- void init_textures (SDL_Renderer ∗renderer, textures_t ∗textures)

  La fonction initialise les texures.
- void apply_sprite (SDL_Renderer ∗renderer, textures_t ∗textures, world_t ∗world)

  La fonction applique la texture du sprite sur le renderer lié à l'écran de jeus.
- void apply_background (SDL_Renderer ∗renderer, textures_t ∗textures)

La fonction applique la texture du fond sur le renderer lié à l'écran de jeu.

- void refresh_graphics (SDL_Renderer ∗renderer, world_t ∗world, textures_t ∗textures)

  La fonction rafraichit l'écran en fonction de l'état des données du monde.
- void clean (SDL_Window ∗window, SDL_Renderer ∗renderer, textures_t ∗textures, world_t ∗world)

  fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des textures, nettoyage des données
- void init (SDL_Window ∗∗window, SDL_Renderer ∗∗renderer, textures_t ∗textures, world_t ∗world)

  fonction qui initialise le jeu: initialisation de la partie graphique (SDL), chargement des textures, initialisation des données
- int main (int argc, char ∗args[ ])

  programme principal qui implémente la boucle du jeu

4.1.1 Detailed Description

Programme principal initial du niveau 0.

Author

Mathieu Constant

Version

1.0

Date

18 mars 2020

4.1.2 Function Documentation

4.1.2.1 apply_background()

```
void apply_background (
            SDL_Renderer * renderer,
            textures_t * textures )
```

La fonction applique la texture du fond sur le renderer lié à l'écran de jeu.

Parameters

renderer	le renderer
textures	les textures du jeu

4.1.2.2 apply_sprite()

```
void apply_sprite (
            SDL_Renderer * renderer,
            textures_t * textures,
            world_t * world )
```

La fonction applique la texture du sprite sur le renderer lié à l'écran de jeus.

Parameters

renderer	le renderer
textures	les textures du jeu
world	le monde du jeu

4.1.2.3 clean()

```
void clean (
            SDL_Window * window,
            SDL_Renderer * renderer,
            textures_t * textures,
            world_t * world )
```

fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des textures, nettoyage des données

Parameters

window	la fenêtre du jeu
renderer	le renderer
textures	les textures
world	le monde

4.1.2.4 clean_data()

```
void clean_data (
            world_t * world )
```

La fonction nettoie les données du monde.

Parameters

world	les données du monde

4.1.2.5 clean_textures()

```
void clean_textures (
            textures_t * textures )
```

La fonction nettoie les textures.

Parameters

textures	les textures

4.1.2.6 handle_events()

```
void handle_events (
            SDL_Event * event,
            world_t * world )
```

La fonction gère les évènements ayant eu lieu et qui n'ont pas encore été traités.

Parameters

event	paramètre qui contient les événements
world	les données du monde

4.1.2.7 init()

```
void init (
            SDL_Window ** window,
            SDL_Renderer ** renderer,
            textures_t * textures,
            world_t * world )
```

fonction qui initialise le jeu: initialisation de la partie graphique (SDL), chargement des textures, initialisation des données

Parameters

window	la fenêtre du jeu
renderer	le renderer
textures	les textures
wordl	le monde

4.1.2.8 init_data()

```
void init_data (
            world_t * world )
```

La fonction initialise les données du monde du jeu.

Parameters

world	les données du monde

< Initialisation de la position x du sprite dans le monde

< Initialisation de la position y du sprite dans le monde

< Initialisation de la largeur du sprite dans le monde

< Initialisation de la hauteur du sprite dans le monde

< Initialisation de l'état de gameover (fin du jeu) à 0

4.1.2.9 init_sprite()

```
void init_sprite (
            sprite_t * sprite,
            int x,
            int y,
            int w,
            int h )
```

La fonction initialise la position du sprite.

Parameters

sprite	le sprite du jeu
x	la position x du sprite
y	la position y du sprite
w	la largeur du sprite
h	la hauteur du sprite

4.1.2.10 init_textures()

```
void init_textures (
            SDL_Renderer * renderer,
            textures_t * textures )
```

La fonction initialise les texures.

Parameters

screen	la surface correspondant à l'écran de jeu
textures	les textures du jeu

4.1.2.11 is_game_over()

```
int is_game_over (
            world_t * world )
```

La fonction indique si le jeu est fini en fonction des données du monde.

Parameters

world	les données du monde

Returns

1 si le jeu est fini, 0 sinon

4.1.2.12 print_sprite()

```
void print_sprite (
            sprite_t * sprite )
```

Fonction de test des coordonnées du sprite.

< Affichage de la position x du sprite

< Affichage de la position y du sprite

< Affichage de la largeur du sprite

< Affichage de la hauteur du sprite

4.1.2.13 refresh_graphics()

```
void refresh_graphics (
            SDL_Renderer * renderer,
            world_t * world,
            textures_t * textures )
```

La fonction rafraichit l'écran en fonction de l'état des données du monde.

Parameters

renderer	le renderer
world	les données du monde
textures	les textures du jeu

4.1.2.14 update_data()

```
void update_data (
            world_t * world )
```

La fonction met à jour les données en tenant compte de la physique du monde.

Parameters

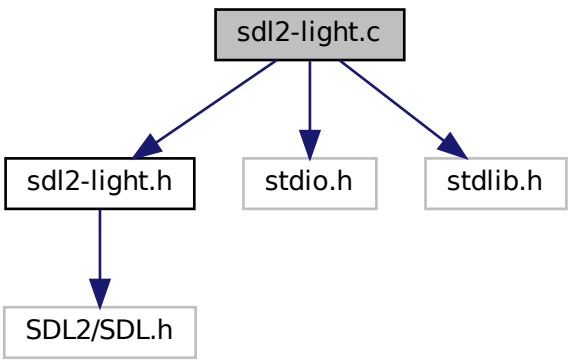
les	données du monde

4.2 sdl2-light.c File Reference

sur-couche de SDL2 pour simplifier son utilisation pour le projet

```
#include "sdl2-light.h"
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for sdl2-light.c:



Functions

- int init_sdl (SDL_Window **window, SDL_Renderer **renderer, int width, int height)

  La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.
- SDL_Texture * load_image (const char path[ ], SDL_Renderer *renderer)

  La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.
- void apply_texture (SDL_Texture *texture, SDL_Renderer *renderer, int x, int y)

  La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.
- void clean_texture (SDL_Texture *texture)

  La fonction nettoie une texture en mémoire.
- void clear_renderer (SDL_Renderer *renderer)

  La fonction vide le contenu graphique du renderer lié à l'écran de jeu.
- void update_screen (SDL_Renderer *renderer)

  La fonction met à jour l'écran avec le contenu du renderer.
- void pause (int time)

  La fonction met le programme en pause pendant un laps de temps.
- void clean_sdl (SDL_Renderer *renderer, SDL_Window *window)

  La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

4.2.1 Detailed Description

sur-couche de SDL2 pour simplifier son utilisation pour le projet

Author

Mathieu Constant

Version

0.2

Date

10 mars 2021

4.2.2 Function Documentation

4.2.2.1 apply_texture()

```
void apply_texture (
            SDL_Texture * texture,
            SDL_Renderer * renderer,
            int x,
            int y )
```

La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.

Parameters

texture	la texture que l'on va appliquer
renderer	le renderer qui va recevoir la texture
x	l'abscisse sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)
y	l'ordonnée sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)

4.2.2.2 clean_sdl()

```
void clean_sdl (
            SDL_Renderer * renderer,
            SDL_Window * window )
```

La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

Parameters

renderer	le renderer à nettoyer
window	la fenêtre à nettoyer

4.2.2.3 clean_texture()

```
void clean_texture (
            SDL_Texture * texture )
```

La fonction nettoie une texture en mémoire.

Parameters

texture	la texture à nettoyer

4.2.2.4 clear_renderer()

```
void clear_renderer (
            SDL_Renderer * renderer )
```

La fonction vide le contenu graphique du renderer lié à l'écran de jeu.

Parameters

renderer	le renderer de l'écran

4.2.2.5 init_sdl()

```
int init_sdl (
             SDL_Window ** window,
             SDL_Renderer ** renderer,
             int width,
             int height )
```

La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.

Parameters

window	la fenêtre du jeu
renderer	le renderer
width	largeur de l'écran de jeu
height	hauteur de l'écran de jeu

Returns

-1 en cas d'erreur, 0 sinon

4.2.2.6 load_image()

```
SDL_Texture* load_image (
             const char path[],
             SDL_Renderer * renderer )
```

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

Parameters

path	est le chemin du fichier image. Le fichier doit être obligatoirement du BMP.
renderer	le renderer

Returns

la surface SDL contenant l'image avec la couleur RGB (255,0,255) rendue transparente. Elle renvoie NULL si le chargement a échoué (ex. le fichier path n'existe pas)

4.2.2.7 pause()

```
void pause (
             int time )
```

La fonction met le programme en pause pendant un laps de temps.

Parameters

time	ce laps de temps en milliseconde

4.2.2.8 update_screen()

```
void update_screen (
            SDL_Renderer * renderer )
```

La fonction met à jour l'écran avec le contenu du renderer.

Parameters

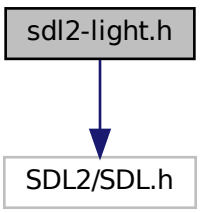
renderer	le renderer de l'écran

4.3 sdl2-light.h File Reference

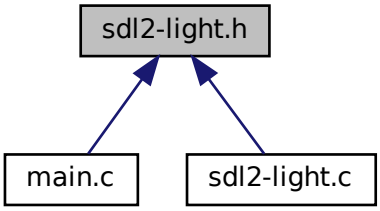
en-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet

```
#include <SDL2/SDL.h>
```

Include dependency graph for sdl2-light.h:



This graph shows which files directly or indirectly include this file:



Functions

- void clean_sdl (SDL_Renderer ∗renderer, SDL_Window ∗window)

  La fonction nettoie le renderer et la fenêtre du jeu en mémoire.
- SDL_Texture ∗ load_image (const char path[ ], SDL_Renderer ∗renderer)

  La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.
- int init_sdl (SDL_Window ∗∗window, SDL_Renderer ∗∗renderer, int width, int height)

  La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.
- void clean_texture (SDL_Texture ∗texture)

  La fonction nettoie une texture en mémoire.
- void apply_texture (SDL_Texture ∗texture, SDL_Renderer ∗renderer, int x, int y)

  La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.
- void clear_renderer (SDL_Renderer ∗renderer)

  La fonction vide le contenu graphique du renderer lié à l'écran de jeu.
- void update_screen (SDL_Renderer ∗renderer)

  La fonction met à jour l'écran avec le contenu du renderer.
- void pause (int time)

  La fonction met le programme en pause pendant un laps de temps.

4.3.1 Detailed Description

en-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet

Author

Mathieu Constant

Version

0.2

Date

10 mars 2021

4.3.2 Function Documentation

4.3.2.1 apply_texture()

```
void apply_texture (
            SDL_Texture * texture,
            SDL_Renderer * renderer,
            int x,
            int y )
```

La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.

Parameters

texture	la texture que l'on va appliquer
renderer	le renderer qui va recevoir la texture
x	l'abscisse sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)
y	l'ordonnée sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)

4.3.2.2 clean_sdl()

```
void clean_sdl (
            SDL_Renderer * renderer,
            SDL_Window * window )
```

La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

Parameters

renderer	le renderer à nettoyer
window	la fenêtre à nettoyer

4.3.2.3 clean_texture()

```
void clean_texture (
            SDL_Texture * texture )
```

La fonction nettoie une texture en mémoire.

Parameters

texture	la texture à nettoyer

4.3.2.4 clear_renderer()

```
void clear_renderer (
            SDL_Renderer * renderer )
```

La fonction vide le contenu graphique du renderer lié à l'écran de jeu.

Parameters

renderer	le renderer de l'écran

4.3.2.5 init_sdl()

```
int init_sdl (
            SDL_Window ** window,
            SDL_Renderer ** renderer,
            int width,
            int height )
```

La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.

Parameters

window	la fenêtre du jeu
renderer	le renderer
width	largeur de l'écran de jeu
height	hauteur de l'écran de jeu

Returns

-1 en cas d'erreur, 0 sinon

4.3.2.6 load_image()

```
SDL_Texture* load_image (
            const char path[],
            SDL_Renderer * renderer )
```

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

Parameters

path	est le chemin du fichier image. Le fichier doit être obligatoirement du BMP.
renderer	le renderer

Returns

la surface SDL contenant l'image avec la couleur RGB (255,0,255) rendue transparente. Elle renvoie NULL si le chargement a échoué (ex. le fichier path n'existe pas)

4.3.2.7 pause()

```
void pause (
            int time )
```

La fonction met le programme en pause pendant un laps de temps.

Parameters

time	ce laps de temps en milliseconde

4.3.2.8 update_screen()

```
void update_screen (
            SDL_Renderer * renderer )
```

La fonction met à jour l'écran avec le contenu du renderer.

Parameters

renderer	le renderer de l'écran

Index

apply_background
    main.c, 11
apply_sprite
    main.c, 11
apply_texture
    sdl2-light.c, 17
    sdl2-light.h, 22

background
    textures_s, 6

clean
    main.c, 12
clean_data
    main.c, 12
clean_sdl
    sdl2-light.c, 18
    sdl2-light.h, 22
clean_texture
    sdl2-light.c, 18
    sdl2-light.h, 22
clean_textures
    main.c, 12
clear_renderer
    sdl2-light.c, 18
    sdl2-light.h, 23

gameover
    world_s, 7

h
    sprite_s, 5
handle_events
    main.c, 13

init
    main.c, 13
init_data
    main.c, 13
init_sdl
    sdl2-light.c, 19
    sdl2-light.h, 23
init_sprite
    main.c, 14
init_textures
    main.c, 14
is_game_over
    main.c, 15

load_image
    sdl2-light.c, 19
    sdl2-light.h, 23

main.c, 9
    apply_background, 11
    apply_sprite, 11
    clean, 12
    clean_data, 12
    clean_textures, 12
    handle_events, 13
    init, 13
    init_data, 13
    init_sprite, 14
    init_textures, 14
    is_game_over, 15
    print_sprite, 15
    refresh_graphics, 15
    update_data, 16

pause
    sdl2-light.c, 19
    sdl2-light.h, 24
print_sprite
    main.c, 15

refresh_graphics
    main.c, 15

sdl2-light.c, 16
    apply_texture, 17
    clean_sdl, 18
    clean_texture, 18
    clear_renderer, 18
    init_sdl, 19
    load_image, 19
    pause, 19
    update_screen, 20
sdl2-light.h, 20
    apply_texture, 22
    clean_sdl, 22
    clean_texture, 22
    clear_renderer, 23
    init_sdl, 23
    load_image, 23
    pause, 24
    update_screen, 24
sprite
    world_s, 7
sprite_s, 5
    h, 5
    w, 5

x, 5
y, 6

textures_s, 6
background, 6

update_data
main.c, 16
update_screen
sdl2-light.c, 20
sdl2-light.h, 24

w
sprite_s, 5
world_s, 7
gameover, 7
sprite, 7

x
sprite_s, 5

y
sprite_s, 6